

AI Tools for Managing Infodemics

Health Misinformation in Video-based Social Media

Project Management Document

Spring 2026 | University of Illinois Urbana-Champaign | CHIME Lab

Instructor: Dr. Brooks

Team

Pranav CR (Lead Engineer)

Aanchal Malhotra | Soobin Jang (Task 3 — Modality Comparison)

Anusha Namburu | Anjali Penamutcha | Shreya Srinivasan (Task 4 — Model Disagreement)

Upasana | Suha | Ananya (Task 5 — Claim-Level Classification)

1. Project Overview

Project Name	WHO Infodemic Monitor
Course	AI Tools for Managing Infodemics (Health Misinformation in Video-based Social Media)
Duration	February 2026 — April 2026 (Spring 2026)
Team Size	12 members across 4 workstreams
Deliverable	Production cloud-deployed AI platform + 3 research tracks
Target Users	WHO representatives, public health researchers, and analysts

The WHO Infodemic Monitor is a production-grade AI system for detecting health misinformation in short-form video (TikTok, Instagram Reels, YouTube Shorts). The system ingests a video, runs it through a transcription and LLM reasoning pipeline, grounds the verdict in PubMed peer-reviewed literature, and returns a structured, explainable classification result. Three parallel research tracks investigated pipeline design, model reliability, and classification granularity to inform future development.

2. Team Structure and Responsibilities

Member(s)	Role	Responsibilities
Pranav CR	Lead Engineer	Full-stack system design and implementation; cloud deployment (Vercel, Railway, Supabase, Upstash); LLM pipeline; PubMed grounding; authentication; CI/CD; architecture decisions; team coordination
Aanchal Malhotra Soobin Jang	Task 3 Research	Designed and ran the modality comparison experiment (audio-only vs visual-only vs multimodal); collected and processed 5 TikTok test videos; generated per-video label and confidence analysis; produced production pipeline recommendation
Anusha Namburu Anjali Penamutcha Shreya Srinivasan	Task 4 Research	Analyzed Llama 2, Mistral, and Gemma model agreement/disagreement on health misinformation classification; designed controlled evaluation protocol; produced agreement and disagreement matrices; identified production routing implications
Upasana Suha Ananya	Task 5 Research	Explored claim-level classification as an alternative to video-level labels; tested LLM-based claim extraction and per-claim JSON output structure; evaluated consistency and interpretability; documented feasibility findings and future direction

3. Project Timeline

Period	Milestone	Description
Feb Week 1	Project Kickoff	Project scope defined; team workstreams assigned; old Docker-based prototype reviewed and audited for reusable components; new cloud-native repository initialized
Feb Week 2-3	Core Pipeline	Config hardening (Slice A); Supabase Storage backend (Slice B); async PubMed grounding with parallel claim queries (Slice C); worker hardening with typed exceptions (Slice D)
Feb Week 4	Auth & Health	Supabase JWT authentication middleware (Slice E); expanded /health endpoint checking DB, Redis, storage, and inference provider (Slice F)
Mar Week 1	Deployment Config	Railway deployment config for API and worker services; Vercel frontend setup; Supabase Auth integration; authenticated API client (Slice G). Parallel: Task 3 and Task 4 research begins
Mar Week 2-3	Frontend + Research	Job history dashboard and result detail pages (Slice H). Task 3: modality comparison experiment running on 5 TikTok videos. Task 4: controlled model evaluation protocol designed and executed
Mar Week 4	Research Analysis	Task 3: results analyzed; production pipeline recommendation formalized. Task 4: agreement/disagreement matrices generated; routing implications documented. Task 5: claim-level classification feasibility testing begins
Apr Week 1-2	Production Deploy	Full end-to-end cloud deployment completed: Vercel (frontend), Railway (API), Supabase (DB + Storage + Auth), Upstash Redis (broker). Database migrations run against production Supabase. System validated on 4 live videos across all 4 classification labels
Apr Week 3	Validation & Docs	End-to-end pipeline validated in production. PubMed grounding confirmed working for MISINFO results. Presentation deck prepared (14 slides). End of semester report drafted. HANDOFF.md written for incoming teams
Apr Week 4	Final Submission	All deliverables finalized: production system, presentation, end of semester report, project management document. Codebase credentials scrubbed and pushed to school GitHub

4. Work Breakdown Structure

4.1 Production System (Pranav CR)

Slice	Component	Deliverable
A	Config Hardening	All tunable values centralized in config.py; Alembic parameterized; migration race condition fixed; CELERY_CONCURRENCY, WHISPER_MODEL_SIZE env vars added
B	Supabase Storage	SupabaseBackend implementation; video upload to cloud bucket; worker downloads to temp file; migration 0002 for storage_key column

C	Async PubMed Grounding	ground_transcript() converted to async; httpx replacing requests; asyncio.gather for parallel claim queries; Ollama dependency removed
D	Worker Hardening	Typed exception hierarchy (StorageError, TranscriptionError, etc.); get_whisper_model() singleton; worker_ready signal for model pre-loading; error_code on Job model
E	Auth Middleware	Supabase JWT verification; REQUIRE_AUTH flag; protected routes; CORS configuration
F	Health Check	GET /health endpoint verifying DB, Redis, storage, and inference provider; 503 on unhealthy status
G	Deployment Config	railway.toml; Dockerfile slim build; Next.js Supabase Auth; middleware.ts; authenticated apiFetch; login page
H	Job Dashboard	GET /jobs endpoint; job history list page; result detail page; 5-second polling; PubMed citation display

4.2 Research Track 3 — Modality Comparison (Aanchal Malhotra, Soobin Jang)

Task	Deliverable
Dataset Collection	5 TikTok videos collected across factual and misinformation content categories
Experiment Design	Controlled 3-mode evaluation: audio-only, visual-only (EasyOCR), multimodal — same Mistral 7B model and prompt
Results Analysis	Per-video label and confidence comparison; latency measurement per mode; 0% label change finding documented
Production Recommendation	Audio-first pipeline with conditional OCR trigger at confidence < 0.60; cost-accuracy tradeoff analysis
Report	End of semester research report submitted; results integrated into team presentation (slides 9-10)

4.3 Research Track 4 — Model Disagreement Analysis (Anusha Namburu, Anjali Penamutcha, Shreya Srinivasan)

Task	Deliverable
Dataset Preparation	New TikTok videos collected with factual and misinformation ground truth labels; Whisper + OCR preprocessing applied
Evaluation Protocol	Controlled inference across Llama 2, Mistral, and Gemma under identical prompt, temperature, and preprocessing conditions
Agreement Matrices	Pairwise model agreement matrix (1.0 = complete agreement) and disagreement matrix (0.0 = no disagreement) generated for Llama 2 x Mistral
Production Implications	Agreement-based routing signal identified: high-agreement cases route to automated output, disagreement cases flagged for human review

Report	End of semester research report submitted; results integrated into team presentation (slides 11-12)
--------	---

4.4 Research Track 5 — Claim-Level Classification (Upasana, Suha, Ananya)

Task	Deliverable
Workflow Design	Claim extraction prompt designed to decompose video transcripts into individual health assertions with per-claim JSON output
Feasibility Testing	LLM-based claim extraction tested across sample videos; outputs evaluated for consistency, interpretability, and alignment with video-level verdict
Findings	Claim-level outputs improve explainability by linking decisions to individual statements; formal evaluation against labeled ground truth identified as next step
Report	Feasibility report submitted; findings documented as future direction in team end of semester report

5. Key Architecture and Project Decisions

Decision	Choice Made	Rationale
Inference Provider	Anthropic Claude (cloud); Ollama Mistral (local research)	Cloud API removes GPU dependency for production; local models kept for research experiments
Transcription	OpenAI Whisper-1 (cloud) / faster-whisper (local)	API-based avoids ML dependency in Docker image; local option preserved for GPU environments
Database	Supabase PostgreSQL	Managed, no-ops, free tier sufficient; Row Level Security available for future multi-tenancy
File Storage	Supabase Storage	Same vendor as DB; S3-compatible; avoids additional vendor; free tier covers project needs
Message Broker	Upstash Redis (serverless)	No always-on cost; pay-per-request; TLS-enabled; compatible with Celery over redis://
Frontend Framework	Next.js 14 + Supabase SSR	Native auth integration; server components reduce API round trips; Vercel deployment is zero-config
Worker Architecture	Celery with solo pool (Windows) / prefork (Linux)	solo pool required on Windows to avoid prefork issues; prefork used on Railway Linux container

OCR Strategy	Disabled by default; conditional trigger planned	Task 3 findings show 0% accuracy benefit from always-on OCR; trigger at confidence < 0.60 recommended
Classification Granularity	Video-level (current); claim-level (future)	Video-level sufficient for MVP; Task 5 work establishes pathway to per-claim verdicts
Evidence Grounding	PubMed E-utilities API	Free, no key required for low volume; returns peer-reviewed citations credible for WHO context

6. Deliverables Summary

Deliverable	Owner	Completed	Status
Production system (cloud-deployed)	Pranav CR	April 2026	Complete
End-to-end pipeline validation (4 videos)	Pranav CR	April 2026	Complete
Presentation deck (14 slides)	Full team	April 2026	Complete
Task 3 — Modality comparison report	Aanchal, Soobin	April 2026	Complete
Task 4 — Model disagreement report	Anusha, Anjali, Shreya	April 2026	Complete
Task 5 — Claim-level classification report	Upasana, Suha, Ananya	April 2026	Complete
End of semester report (17 pages)	Full team	April 2026	Complete
Project management document	Pranav CR	April 2026	Complete
HANDOFF.md setup guide for incoming teams	Pranav CR	April 2026	Complete
Codebase pushed to school GitHub	Pranav CR	April 2026	Complete

7. Tools and Technologies Used

Category	Tool / Service	Purpose
Project Planning	CLAUDE.md + Slices A-H	Living architecture document and implementation slice specs served as the project plan and decision log
Version Control	GitHub	Source code management; separate personal repo (dev) and school repo (submission)
Communication	Email / Teams	Team coordination; professor and stakeholder updates

Backend	FastAPI + Celery	REST API and async task processing
Frontend	Next.js 14	Web interface for WHO researchers
Database	Supabase PostgreSQL	Job, video, and result storage
Cloud Deploy	Vercel + Railway	Frontend and backend hosting
Message Queue	Upstash Redis	Celery broker for async job dispatch
LLM Inference	Anthropic Claude	Video classification and claim extraction
Transcription	OpenAI Whisper-1	Audio-to-text conversion
Evidence	PubMed E-utilities	Peer-reviewed citation retrieval for grounding
Research Compute	Local GPU (GTX 1650)	Llama 2, Mistral, Gemma inference for research tracks
Dev Environment	VS Code + Claude Code CLI	Implementation and code generation

8. Risks and Mitigations

Risk	Impact	Mitigation
Railway free tier — single service limit	High	Worker runs locally pointing at cloud Redis; upgrade to Hobby (\$5/mo) planned when budget approved
OpenAI API quota exhausted	High	Switched to Anthropic Claude as primary inference provider; OpenAI retained as fallback via env var
Supabase shared project with existing tables	Medium	Confirmed no table name collisions before running migrations; plan to migrate to dedicated project
Whisper hallucination on noisy audio	Medium	VAD pre-segmentation via faster-whisper; CANNOT_RECOGNIZE label catches low-signal videos
PubMed rate limiting on parallel queries	Low	asyncio.gather with staggered delays ($i * 0.5s$); best-effort grounding returns [] on failure
Personal credentials in codebase	High	All secrets in gitignored .env files; .env.example with placeholders committed; HANDOFF.md documents setup
Small research track datasets	Medium	Explicitly flagged as limitation in all research reports; larger dataset testing identified as future work

9. Future Work and Roadmap for Incoming Teams

The items below represent concrete next steps for any team that picks up this project. They are ordered within each priority level by impact. Items marked High should be addressed before expanding access to external WHO researchers. Medium items improve the system's accuracy and usability. Low items are valuable but not blocking.

Item	Category	Priority	Where to Start
Full Cloud Worker Deployment	Infrastructure	High	railway.toml (worker service already defined), Railway dashboard
URL Ingestion	Feature	High	backend/app/api/routers/videos.py, backend/app/services/video_service.py
Dedicated Supabase Project	Infrastructure	High	backend/.env, Supabase dashboard
Per-User Job Isolation	Feature	High	backend/app/db/models/job.py, backend/app/services/job_service.py (new migration 0005 needed)
OCR Re-enable with Confidence Trigger	Pipeline	Medium	backend/app/core/extraction/multimodal.py, backend/app/core/extraction/ocr/text_extractor.py
Stronger PubMed Evidence Search	Pipeline	Medium	backend/app/core/grounding/pubmed.py
Three-way LLM Ensemble Routing	Pipeline	Medium	backend/app/core/inference/__init__.py, backend/app/worker/tasks.py
Confidence Calibration and Eval Harness	Research	Medium	New scripts/eval_pipeline.py (does not exist yet)
Claim-Level Classification and UI	Feature	Medium	backend/app/core/pipeline/__init__.py, frontend/app/jobs/[job_id]/JobDetailClient.tsx
Webhook and Async Notification	Feature	Low	backend/app/worker/tasks.py (emit event on completion), frontend (subscribe to Supabase Realtime)
Rate Limiting and Abuse Prevention	Infrastructure	Low	backend/app/api/routers/videos.py, requirements.txt (add slowapi)
Language Support	Feature	Low	backend/app/core/extraction/transcription.py, backend/app/core/inference/providers/anthropic_provider.py

9.1 High Priority

Full Cloud Worker Deployment

Move the Celery analysis worker from local machine to Railway. The worker service config already exists in railway.toml. Requires Railway Hobby plan (\$5/month) to run a second service alongside the API. This is the

single most important infrastructure change — without it the system stops processing jobs whenever the developer's machine is off.

Where to start: [railway.toml](#) (worker service already defined), [Railway dashboard](#)

URL Ingestion

Accept a TikTok, Instagram Reels, or YouTube Shorts URL directly instead of requiring researchers to download and re-upload a video file. The system would fetch the video server-side using yt-dlp. The upload endpoint and video_service.py would need to handle a URL input as an alternative to a multipart file upload.

Where to start: [backend/app/api/routers/videos.py](#), [backend/app/services/video_service.py](#)

Dedicated Supabase Project

The current deployment shares a free-tier Supabase project with other personal projects, which has storage limits and no isolation. A dedicated Pro project (\$25/month) provides proper separation, more storage capacity, and better performance. Migration requires a pg_dump of the current data, updating the DATABASE_URL and DATABASE_URL_SYNC env vars, and running alembic upgrade head against the new project. No code changes needed.

Where to start: [backend/.env](#), [Supabase dashboard](#)

Per-User Job Isolation

Currently all authenticated users see all jobs in the job history dashboard. Adding a user_id column (from the JWT sub field) to the Job model and filtering GET /jobs by the current user's sub would give each researcher their own isolated view. This is a small change with significant multi-tenancy impact.

Where to start: [backend/app/db/models/job.py](#), [backend/app/services/job_service.py](#) (new migration 0005 needed)

9.2 Medium Priority

OCR Re-enable with Confidence Trigger

OCR is currently disabled. Task 3 research found 0% accuracy benefit from always-on OCR, so it should be re-enabled with a conditional trigger: run EasyOCR only when audio transcription confidence falls below 0.60. The OCR code in text_extractor.py and multimodal.py is complete — it just needs to be wired back in with the confidence threshold check in the pipeline.

Where to start: [backend/app/core/extraction/multimodal.py](#), [backend/app/core/extraction/ocr/text_extractor.py](#)

Stronger PubMed Evidence Search

The current grounding step uses simple keyword queries generated by the LLM. Improvements: (a) MeSH term queries for medical precision, (b) hybrid BM25 and dense retrieval instead of keyword-only search, (c) abstract-level NLI verification to confirm the retrieved paper actually supports or refutes the claim rather than just matching keywords. This would bring citation quality closer to the MEGA-RAG benchmark.

Where to start: [backend/app/core/grounding/pubmed.py](#)

Three-way LLM Ensemble Routing

Task 4 found full agreement between Llama 2 and Mistral on clear cases. The production implication is an ensemble routing layer: run all three models on low-confidence cases, route based on majority vote, and flag cases where models disagree for human review. The provider abstraction in core/inference/ already supports multiple providers simultaneously.

Where to start: [backend/app/core/inference/__init__.py](#), [backend/app/worker/tasks.py](#)

Confidence Calibration and Eval Harness

Current confidence scores are LLM self-assessments, not calibrated probabilities. Build an evaluation harness: collect a labeled dataset of health videos with ground truth labels, run the full pipeline, and compute precision, recall, and F1 per label. Use this to set better confidence thresholds, detect systematic model blind spots, and measure accuracy improvements as the pipeline evolves.

Where to start: [New scripts/eval_pipeline.py](#) (does not exist yet)

Claim-Level Classification and UI

Task 5 established that per-claim verdicts are feasible and improve explainability. The next steps are: integrate claim-level outputs with the PubMed grounding step so each extracted claim generates its own citation retrieval query, and surface claim breakdowns in the frontend result page as a collapsible drill-down below the video-level verdict. Each claim would show its own label, confidence, and linked paper.

Where to start: [backend/app/core/pipeline/__init__.py](#), [frontend/app/jobs/\[job_id\]/JobDetailClient.tsx](#)

9.3 Low Priority

Webhook and Async Notification

Currently users must keep the browser open and poll every 5 seconds for job completion. Adding email or webhook notification when a job completes would make the system usable for longer analysis queues. Supabase Realtime subscriptions could push job status updates to the frontend without polling, eliminating the current polling architecture.

Where to start: [backend/app/worker/tasks.py](#) (emit event on completion), [frontend](#) (subscribe to Supabase Realtime)

Rate Limiting and Abuse Prevention

Before sharing access with external WHO researchers, add rate limiting to the upload endpoint (for example, a maximum of 10 videos per user per hour). This can be implemented using slowapi or a Redis-backed counter. Without it, a single user could exhaust API credits or storage capacity.

Where to start: [backend/app/api/routers/videos.py](#), [requirements.txt](#) (add slowapi)

Language Support

Whisper supports multilingual transcription out of the box. The LLM classification prompts need to be tested and tuned for Spanish, Portuguese, Hindi, and Arabic content — the top languages for health misinformation in WHO priority regions. Start by testing with Spanish-language TikToks and evaluating whether Claude's classification quality holds without prompt changes.

Where to start: [backend/app/core/extraction/transcription.py](#), [backend/app/core/inference/providers/anthropic_provider.py](#)